National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing

Class: BSCS-2023-CD

Course: CS-346 Compiler Construction

Lab Project : Compiler Construction

Date: 30, April 2026

Lab Instructor: Ms. Urooj Akmal

Instructor: Miss Yusra

Project Overview

This project requires you to build a complete mini-compiler from scratch, progressing through Labs 06 to 12 of CS-346. Each lab introduces one phase of the compiler. You will integrate every phase into a single unified pipeline, finishing with a fully working system capable of tokenising, parsing, type-checking, generating intermediate code, optimising it, and producing LLVM IR..

1.1 Project Goals

- Build each compiler phase as a self-contained, modular component.
- Integrate all phases into a single, cohesive compiler pipeline.
- Demonstrate understanding of each phase through working code and clear documentation.
- Apply optimisation transformations and compare performance before and after.
- Generate LLVM IR and leverage Clang for final code generation.

1.2 Group Policy

Groups of 3 to 4 students are allowed. Each group submits one unified codebase. Every member is expected to contribute to all modules and must be able to answer viva questions on any part of the project.

> **Note:** *Solo or pairs are discouraged; groups larger than 4 are not permitted.*

1.3 Technology Stack

Tool / Technology	Role in Project
Flex (win_flex on Windows)	Lexical Analyser Generator
Bison (win_bison on Windows)	Parser Generator (LALR(1))
GCC / G++	Compiling generated C/C++ source files
LLVM + Clang	Code generation and IR optimisation
C / C++	Implementation language for semantic analysis and IR
Linux / Windows (VM)	Development platform

1.4 Compiler Pipeline

The diagram below shows the full pipeline you are required to implement:

Full Compiler Pipeline
Source Code (.l / .y / .c input)
↓
[Module 1] Lexical Analysis (Flex) → Token Stream
↓
[Module 2] Syntax Analysis / Parsing (Bison) → Parse Tree
↓
[Module 3] Extended Grammar (Functions, Exponentiation, Math)
↓
[Module 4] First & Follow Sets → LL(1) Parsing Table
↓
[Module 5] Semantic Analysis: Type Checking + Scope Checking
↓
[Module 6] Intermediate Representation (Three-Address Code)
↓
[Module 7] Code Optimisation (Constant Folding, CSE, Dead Code, Loop Opts)
↓
[Module 8] LLVM IR Generation via Clang → Optimised Native Binary

Module 1 — Lexical Analysis (Flex)

The lexer is the first phase of the compiler. It reads raw source text and converts it into a stream of tokens that the parser can process. Using Flex, you define token patterns with regular expressions in a .l file.

2.1 Background

A lexer (also called a scanner or tokeniser) scans the input character by character, matches patterns, discards whitespace and comments, and emits tokens. The Lab 05 tasks serve as the starting point for this module.

2.2 Lab 05 Baseline Requirements

Your lexer must satisfy all requirements from Lab 05. These are the minimum baseline rules — do not add complexity beyond what is described here for this module:

Task 1 — Lexer

- Match integer literals (e.g. 42, 0, 100).
- Match floating-point literals (e.g. 3.14, 0.5).
- Match identifiers: start with a lowercase letter; digits allowed in non-initial positions.
- **Keywords:** Recognise keywords:
  - `if, then, begin, end, procedure, function`
- Recognise arithmetic operators: + – * /
- Skip whitespace (spaces, tabs, newlines).
- Detect and print any unrecognised character with an error message.

Task 2 — Postfix Expression Evaluator

- Accept non-negative integer operands.
- Support operators: + – *
- Evaluate the postfix expression using a stack.
- Display stack transitions at each step (push/pop).
- Print the final result.

2.3 Extended Token Set (Full Project)

For the full pipeline, your lexer must additionally handle the following. Extend your Lab 05 lexer rather than rewriting it:

Token Category	Examples	Flex Pattern
Keywords	int, float, while, return, else	"int"|"float"|"if"|"else"|"while"|"return"
Identifiers	x, myVar, result2	[a-zA-Z_][a-zA-Z0-9_]*
Integer Literals	42, 0, 100	[0-9]+
Float Literals	3.14, 0.5, 2.0	[0-9]+\.[0-9]+
Operators	+, -, *, /, ^, =, ==, !=	Single or double character tokens
Delimiters	( ) { } ; ,	Single character tokens
Comments	// ... and /* ... */	Discard — no token returned
Whitespace	spaces, tabs, newlines	Discard — no token returned

2.4 Build Commands

Linux

```
flex lexer.l
gcc -o lexer lex.yy.c -lfl
./lexer < input.txt
```

Windows

```
win_flex lexer.l
gcc -o lexer lex.yy.c
lexer.exe < input.txt
```

2.5 Deliverables

1. A .l file implementing all token categories listed above.
2. Sample input file and a screenshot of token output.
3. Integration of the lexer with the Bison parser (Module 2).

Module 2 — Syntax Analysis / Parsing (Bison)

The parser receives the token stream from the lexer and verifies that it conforms to the language grammar. Bison generates an LALR(1) parser from a .y grammar specification file.

3.1 Background

Syntax analysis determines whether the sequence of tokens forms a valid sentence in the language. Context-Free Grammars (CFGs) written in BNF specify the language. Parse trees show the full derivation; abstract syntax trees (ASTs) are compact representations that omit redundant nodes.

3.2 Required Calculator Variants

You must implement all three expression notations:

Notation	Example	Description
Postfix (given)	4 8 +	Operator follows operands; stack-based evaluation
Prefix	+ 4 8	Operator precedes operands; recursive evaluation
Infix	4 + 8	Standard notation; requires precedence / associativity rules

3.3 Key Grammar Concepts Required

- Declare operator precedence and associativity using %left, %right, and %nonassoc in your .y file.
- Support at minimum: + – * / ^ ( )
- Parse trees must show the full derivation; syntax trees must eliminate redundant nodes.
- Tree traversal: pre-order (prefix), in-order (infix), post-order (postfix).

3.4 Build Commands (Infix Example)

Linux

```
bison -d infix.y
flex calc_lex.l
gcc -o PARSER lex.yy.o infix.tab.o -lfl -lm
```

Windows

```
win_bison -d infix.y
win_flex calc_lex.l
gcc -o PARSER infix.tab.c lex.yy.c -lm
```

3.5 Deliverables

4. Three .y files (or one combined) for postfix, prefix, and infix parsers.
5. Screenshot of each parser evaluating at least two sample expressions.
6. Integration with Module 1 lexer.

Module 3 — Extended Grammar (Functions)

Extend the basic expression parser to support exponentiation (right-associative) and the mathematical functions log() and exp(), while correctly evaluating floating-point arithmetic.

4.1 Extended Grammar (Required)

```
E  →  E + T  |  E – T  |  T
T  →  T * F  |  T / F  |  F
F  →  B ^ F  |  B              (right-associative exponentiation)
B  →  ( E )  |  id  |  num  |  log( E )  |  exp( E )
```

4.2 Requirements

Requirement	Detail
Input	Arithmetic expressions (integer and floating-point)
Output	Evaluated numerical result
Float Handling	Lexer must convert strings to float/double using atof() or strtod()
Whitespace	Lexer must discard spaces, tabs, and newlines
Error Handling	Any invalid input must produce a descriptive error message
Debug Mode	Use #define YYDEBUG 1 and yydebug = 1 for debug output

4.3 Deliverables

7. Updated .y file with exponentiation and log/exp support.
8. Screenshot showing evaluation of expressions containing log() and exp().

> **Note:** *Use %right for ^ to enforce right-associativity. Link with -lm for math.h functions.*

Module 4 — First & Follow Sets / LL(1) Parsing Table

FIRST and FOLLOW sets are essential for constructing predictive (LL(1)) parsers. You will implement programs to compute these sets and build the LL(1) parsing table from any given grammar.

5.1 Target Grammar

```
E   →  T E'
E'  →  + T E'  |  - T E'  |  ε
T   →  F T'
T'  →  * F T'  |  / F T'  |  ε
F   →  ( E )  |  id  |  num
```

5.2 Expected FIRST and FOLLOW Sets

Non-terminal	Nullable	FIRST	FOLLOW
E	No	{ (, id, num }	{ $,) }
E'	Yes	{ +, –, ε }	{ $,) }
T	No	{ (, id, num }	{ $,), +, – }

T'	Yes	{ *, /, ε }	{ $,), +, – }
F	No	{ (, id, num }	{ *, /, $,), +, – }

5.3 LL(1) Compatibility Requirements

- Grammar must be unambiguous.
- No left recursion (eliminate if present before computing sets).
- No FIRST / FOLLOW conflicts for any non-terminal.

5.4 Required Programs

9. Program to compute FIRST sets from any grammar input.
10. Program to compute FOLLOW sets from any grammar input.
11. Program to construct the LL(1) parsing table from FIRST and FOLLOW sets.

Module 5 — Semantic Analysis (Type & Scope Checking)

Once the AST is built, semantic analysis verifies that the program is logically valid. This includes type checking (compatible types in operations) and scope checking (variables used within their declared scope).

6.1 Task A — Type Checking and Type Casting

- Track the declared type of each variable (int, float, etc.).
- Check that operations are performed on compatible types.
- Report a type error if incompatible types are mixed without explicit casting.
- Perform implicit or explicit type casting where applicable (e.g. int → float promotion).

6.2 Task B — Scope Checking

- Maintain a symbol table with separate entries for global and local scopes.
- When the same identifier exists in both scopes, local scope takes precedence.
- Report an error when a variable is used before its declaration.

6.3 Deliverables

12. Source file(s) implementing the symbol table, type checker, and scope checker.
13. Screenshot showing correct error detection for a type mismatch and an undeclared variable.

> **Note:** *A scope stack is the recommended data structure for managing nested scopes.*

Module 6 — Intermediate Representation (IR Generation)

Generate Three-Address Code (TAC) from the AST produced by semantic analysis. TAC is a simple, low-level representation where each instruction has at most three operands and one operator.

7.1 Required TAC Forms

- Binary operations: x = y op z
- Unary operations: x = op y
- Copy statements: x = y
- Unconditional jump: goto L
- Conditional jump: if x relop y goto L
- Function calls: param x / call f, n / return x
- Array access: x = a[i] and a[i] = x

7.2 Deliverables

14. IR generator source files integrated with Module 5 output.
15. Printed TAC listing for at least one non-trivial input program.

Module 7 — Code Optimisation

Apply optimisation passes to the TAC generated in Module 6. For each technique, print the IR before and after transformation.

8.1 Required Optimisation Techniques (implement at least 3 of 5)

Technique	Description
Constant Folding	Evaluate constant expressions at compile time (e.g. 3+4 → 7)
Constant Propagation	Replace variables known to hold a constant with that constant
Common Sub-expression Elimination	Reuse previously computed results instead of recalculating
Dead Code Elimination	Remove assignments whose values are never subsequently used
Loop Optimisation	Loop-invariant code motion (LICM) and/or loop fusion

8.2 Additional Tasks

- Task 4 — Control Flow: detect and remove unreachable code blocks.
- Task 5 — Loop Optimisation: implement at least one loop technique (LICM recommended).
- Task 6 — Performance Comparison: measure execution time (min 5 runs) before and after optimisation; present results in a table with a speedup ratio.

8.3 Deliverables

16. Optimiser source files with clear before/after IR output for each technique.
17. Performance comparison table in the report.

Module 8 — LLVM and Code Generation

Install LLVM, generate LLVM IR using Clang, and compare optimised versus non-optimised output. Annotate key IR instructions in your report.

9.1 Installation Commands

```
clang --version          # verify Clang is installed
sudo apt-get install clang   # if not installed
```

9.2 Key IR Generation Commands

Command	Output / Purpose
clang -o test test.c	Compile to native executable
clang test.c -S -emit-llvm	Generate unoptimised LLVM IR (.ll file)
clang test.c -S -emit-llvm -O3	Generate optimised LLVM IR
llvm-as test.ll	Assemble IR text to bitcode (.bc)
llvm-dis test.bc	Disassemble bitcode back to text

9.3 Required Analysis

- Generate LLVM IR (unoptimised) for both provided programs and annotate key constructs.
- Generate LLVM IR with -O3 for both programs.
- Compare and document the differences — what did LLVM eliminate or transform?
- Identify at least 3 specific optimisations visible in the -O3 output.
- Explain the semantics of key instructions: alloca, load, store, ret, add, call.

9.4 Deliverables

18. Verified LLVM installation (screenshot of clang --version).
19. .ll files (unoptimised and -O3) for both test programs included in submission.
20. Annotated comparison in the report.

Evaluation Criteria

The project is graded holistically across all modules. Marks are awarded for correctness, completeness, code quality, integration, and documentation. Total marks: 100 (with 6 marks of buffer; final score is capped at 100).

10.1 Mark Distribution

Module / Component	Max	Code	Docs
Module 1: Lexical Analysis (Flex)	8	4	4
Module 2: Parsing – Postfix / Prefix / Infix	8	4	4
Module 3: Extended Grammar (Functions)	8	4	4
Module 4: First & Follow / LL(1) Table	8	4	4
Module 5: Type & Scope Checking	8	4	4
Module 6: IR Generation (TAC)	10	6	4
Module 7 – Task 1: Refactoring	5	3	2
Module 7 – Task 2: IR Before Opt. (Print)	5	3	2
Module 7 – Task 3: Optimisation (3–5 Techs)	15	9	6
Module 7 – Task 4: Control Flow / Unreachable	7	4	3
Module 7 – Task 5: Loop Optimisation	8	5	3
Module 7 – Task 6: Performance Comparison	5	3	2
Module 8: LLVM Installation & IR Generation	5	3	2
Module 8: Optimised vs Non-optimised Analysis	6	2	4
TOTAL	**106***	—	—

** 6 marks of buffer. Final score is capped at 100.*

10.2 Code Correctness Rubric (per module)

Score	Criteria
Full marks	Compiles without errors; correct output on all test cases; handles edge cases
75 %	Compiles and runs; minor incorrect output on edge cases or one test case fails
50 %	Compiles but produces incorrect output on multiple cases, or partial implementation
25 %	Has compile errors or crashes; demonstrates partial understanding
0 %	Not submitted, or copied without understanding

10.3 Integration & Pipeline (overall)

- All modules compile together as a single project — no separate standalone files.
- Compiler pipeline executes end-to-end on a test program (3 marks).
- Clean, modular code structure with separate files per phase (2 marks).
- Makefile or build script provided (2 marks).

10.4 Academic Integrity

> **Warning:** All code must be your own work. Copying or sharing source files between groups is strictly prohibited. Automated plagiarism detection will be applied to all submissions. Credited use of reference materials (e.g. Bison manual, LLVM documentation) is permitted but must be cited. Any violation will result in a zero for the entire project with possible further disciplinary action.

Module 9 — Project Report (Documentation)

Every group must submit a comprehensive PDF report documenting all modules. The report is not an afterthought — it carries dedicated marks within each module's documentation allocation. A well-written report demonstrates that you understand not just how to run the code, but why each decision was made.

11.1 Required Report Structure

#	Section	Required Content
1	**Cover Page**	University name, course, project title, group member names & roll numbers, date
2	**Table of Contents**	Auto-generated or manually listed with page numbers
3	**Introduction**	Brief overview of a compiler and the pipeline you are implementing
4	**Module 1 – Lexer**	Explanation of Flex rules, token table, screenshot of token output
5	**Module 2 – Parser**	Grammar rules used, screenshot of each calculator variant working
6	**Module 3 – Extended Grammar**	Extended grammar listing, screenshot of log/exp evaluation
7	**Module 4 – First & Follow**	Computed FIRST/FOLLOW sets table, LL(1) table, screenshot

8	**Module 5 – Semantic Analysis**	Symbol table design, type/scope error examples, screenshot
9	**Module 6 – IR Generation**	TAC format description, sample TAC output screenshot
10	**Module 7 – Optimisation**	Before/after IR for each technique, performance comparison table
11	**Module 8 – LLVM**	Installation screenshot, annotated .ll files, optimised vs non-optimised comparison
12	**Conclusion**	Summary of what was achieved and any challenges encountered
13	**References**	Cited tools, textbooks, and documentation used

11.2 Documentation Quality Standards

- Every module section must include at least one screenshot of the program running with sample input and correct output.
- Code snippets shown in the report must be syntax-highlighted or presented in a monospace font.
- Before/after IR listings for each optimisation pass must be clearly labelled.
- The performance comparison table (Task 6) must include average times over 5 runs and a speedup ratio column.
- LLVM IR excerpts must be annotated inline explaining what each key instruction does.
- The report must be submitted as PDF, not as a raw Word file.

11.3 Formatting Guidelines

- Minimum 10 pages, no upper limit.
- Font: Arial or Times New Roman, size 11–12 pt for body text.
- Headings must be numbered and consistent throughout.
- All tables and figures must have captions.
- Page numbers must appear on every page except the cover.

11.4 Marks Allocation for Documentation

Documentation marks are embedded within each module's allocation (2-4 marks per module, see Section 10.1). The following criteria apply to all doc marks:

Documentation Criterion	Marks
Screenshot of correct output included for the module	1 mark (per module)

Clear explanation of implementation decisions	0.5 marks (per module)
Before/after IR listed for each optimisation (Task 3)	Included in Task 3 marks
Performance comparison table present (Task 6)	Included in Task 6 marks
LLVM IR annotated with explanations	Included in Module 8 marks

> **Note:** *Your report will be reviewed during the viva. Be prepared to explain anything written in it.*

Submission Instructions

12.1 Deliverable Package

Upload a single ZIP archive to the LMS submission link. The archive must contain:

- Source files directory — all .l, .y, and .c/.cpp/.h files organised by module.
- Makefile or build script that compiles the full project.
- PDF report documenting every module (see Section 11).
- README.txt — brief description of how to build and run the project.

12.2 Recommended Directory Structure

```
project/
  ├── lexer/           # Module 1: .l file(s)
  ├── parser/          # Modules 2 & 3: .y files
  ├── first_follow/    # Module 4: C programs
  ├── semantic/        # Module 5: type & scope checker
  ├── ir/              # Module 6: IR generator
  ├── optimizer/       # Module 7: optimisation passes
  ├── llvm/            # Module 8: LLVM test files & .ll outputs
  ├── Makefile
  ├── README.txt
  └── report.pdf
```

12.3 Submission Deadlines

Module	Lab Date	Submission Deadline
Labs 01–07 (Lexer, Parser, Functions)	Feb – Mar 2026	Per LMS link
Lab 08 (First & Follow)	26 March 2026	Per LMS link
Lab 09 (Type & Scope Checking)	9 April 2026	Per LMS link

Lab 10 (Combining modules & Cleaning code)	16 April 2026	Per LMS link
Lab 11 (Code & Loop Optimisation)	23 April 2026	Per LMS link
Lab 12 (LLVM)	30 April 2026	Per LMS link
Final Integrated Project	TBD by Instructor	Per LMS link

> **Warning:** Late submissions will be penalised as per the course policy in the course outline.